Distributed Database Concepts

Introduction

- Distributed computing system
  - Consists of several processing sites or nodes interconnected by a computer network
  - Nodes cooperate in performing certain tasks
  - Partitions large task into smaller tasks for efficient solving
- Big data technologies
  - Combine distributed and database technologies
  - Deal with mining vast amounts of data

Distributed Database Concepts

- What constitutes a distributed database?
  - Connection of database nodes over computer network
  - Logical interrelation of the connected databases
  - Possible absence of homogeneity among connected nodes
- Distributed database management system (DDBMS)
  - Software system that manages a distributed database

Distributed Database Management System(DDBMS)

- A system that is used for managing the storage and retrieval of data across multiple interconnected databases “Distributed db”.
- In this case, the interconnected databases are situated in different geographical areas.
- In DDBMS, one can access and store data transparently from different geographical locations and ensure high availability, scalability, and fault tolerance mechanisms.
- DDBMSs are designed to handle huge amounts of data spread across different sites. It can give seamless performance in data sharing and collaboration by organizations.

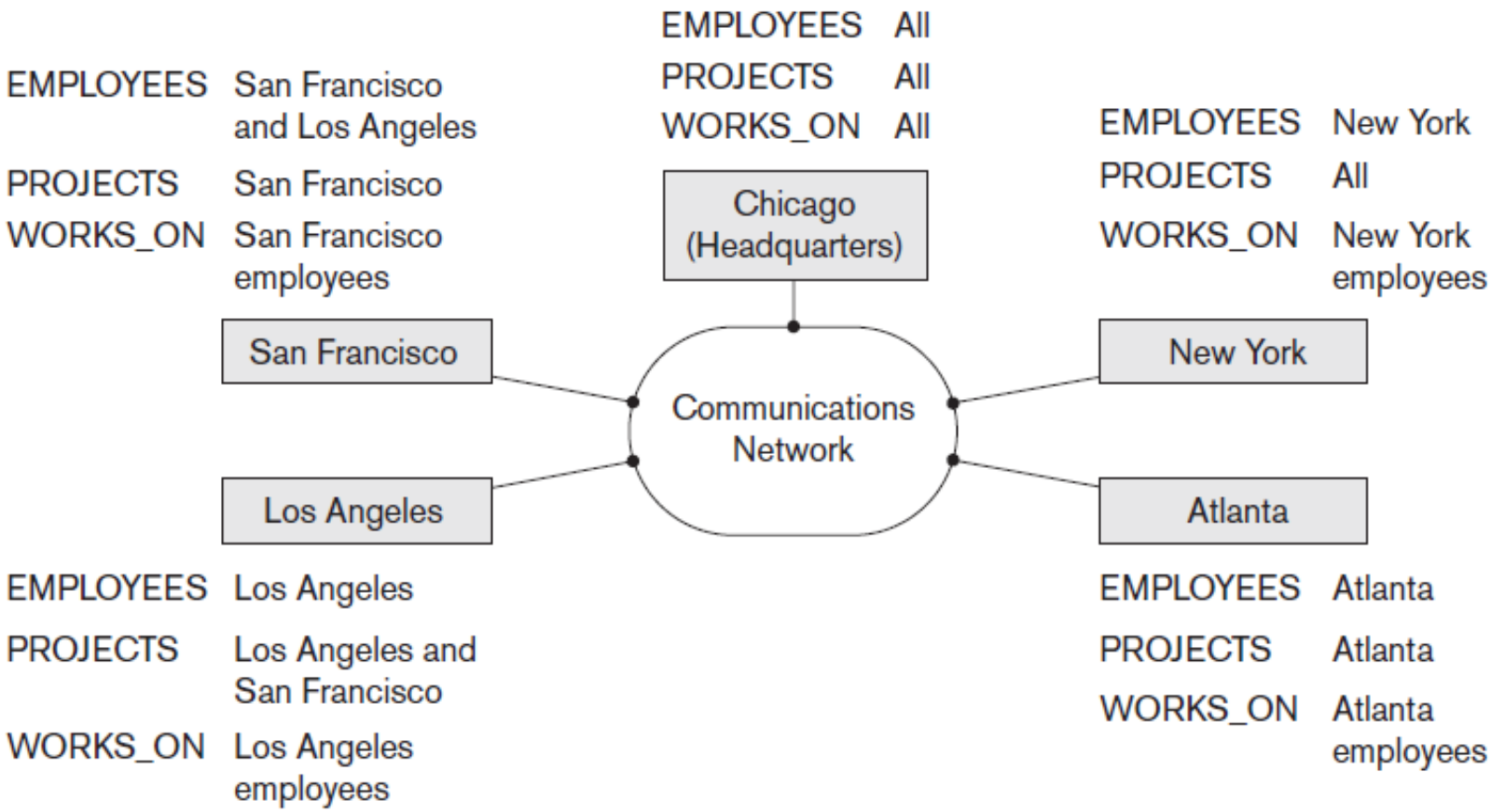
Features of Distributed DBMS

- **Data Fragmentation:** The overall database system is divided into smaller subsets which are fragmentations. This fragmentation can be three types horizontal (divided by rows depending upon conditions), vertical (divided by columns depending upon conditions), and hybrid (horizontal + vertical).
- **Data Replication:** DDBMS maintains and stores multiple copies of the same data in its different fragments to ensure data availability, fault tolerance, and seamless performance.
- **Data Allocation:** It determines if all data fragments are required to be stored in all sites or not. This feature is used to reduce network traffic and optimize the performance.
- **Data Transparency:** DDBMS hides all the complexities from its users and provides transparent access to data and applications to users.

Types of Distributed DBMS

- **Homogeneous:** In this type of DDBMS, all the participating sites should have the exact same DBMS software and architecture which makes all underlying systems consistent across all sites. It provides simplified data sharing and integration.
- **Heterogeneous:** In this type of DDBMS, the participating sites can be from multiple sites and use different DBMS software, data models, or architectures. This model faces little integration problem as all site's data representation and query language can be different from each other.
- **Federated:** Here, the local databases are maintained by individual sites or federations. These local databases are connected via a middleware system that allows users to access and query data from multiple distributed databases. The federation combines different local databases but maintains autonomy at the local level.

Distributed Databases



Data distribution and replication among distributed databases

Autonomy

- Determines extent to which individual nodes can operate independently
- Design autonomy
  - Independence of data model usage and transaction management techniques among nodes
- Communication autonomy
  - Determines the extent to which each node can decide on sharing information with other nodes
- Execution autonomy
  - Independence of users to act as they please

Advantages of Distributed Databases

- Improved ease and flexibility of application development
  - Development at geographically dispersed sites
- Increased availability
  - Isolate faults to their site of origin
- Improved performance
  - Data localization
- Easier expansion via scalability
  - Easier than in non-distributed systems

Data Fragmentation

- Fragmentation is the process of dividing a database table into smaller parts called fragments, which are stored on different sites in a distributed system.
- Each fragment is a logical unit of data, and together, they must allow the **reconstruction** of the original table without any data loss- typically using **UNION** or **JOIN** operations.

- users don’t need to know that the data is fragmented. This is known as **fragmentation transparency**.
- We have three methods for data fragmenting of a table:
- Horizontal fragmentation
- Vertical fragmentation
- Mixed or Hybrid fragmentation

Horizontal fragmentation

- Horizontal fragmentation splits a table by rows based on conditions on one or more attributes. Each **fragment contains a subset of rows**, which are then stored at different sites. This helps in localizing data access.
- In relational algebra, it's represented using the **SELECT (σ)** operation on the table.
- $\sigma_p(T)$
- **σ** is relational algebra operator for selection
- **p** is the condition satisfied by a horizontal fragment

- Example: consider an EMPLOYEE table (T) :

Eno	Ename	Design	Salary	Dep
101	A	abc	3000	1
102	B	abc	4000	1
103	C	abc	5500	2
104	D	abc	5000	2
105	E	abc	2000	2

- This EMPLOYEE table can be divided into different fragments like:
- EMP 1 = $\sigma_{Dep = 1}$ EMPLOYEE
- EMP 2 = $\sigma_{Dep = 2}$ EMPLOYEE

- These two fragments are:
- **1. T1 fragment of Dep = 1**

Eno	Ename	Design	Salary	Dep
101	A	abc	3000	1
102	B	abc	4000	1

- **2. T2 fragment of Dep = 2**

Eno	Ename	Design	Salary	Dep
103	C	abc	5500	2
104	D	abc	5000	2
105	E	abc	2000	2

- here it is possible to get back T as **T = T1 ∪ T2 ∪ .... ∪ TN**

Vertical Fragmentation

- Vertical fragmentation splits a table by columns (attributes), storing different columns on different sites.
- Since each site may not need all columns, this improves efficiency.
- To reconstruct **the** original table, each fragment must include a **common key** (like the primary key or a tuple ID) so that rows can be joined correctly using a **natural JOIN**.
- $\pi_{a1,\ a2,\ldots,\ an}$ (T)
- **π** is relational algebra operator
- **a1…., an** are the attributes of T
- **T** is the table (relation)

- example, for the EMPLOYEE table we have T1 as :

Eno	Ename	Design	Tuple_id
101	A	abc	1
102	B	abc	2
103	C	abc	3
104	D	abc	4
105	E	abc	5

Salary	Dep	Tuple_id
3000	1	1
4000	2	2
5500	3	3
5000	1	4
2000	4	5

- For the second sub table of relation after vertical fragmentation is:
- to get back to the original T, we join these two fragments T1 and T2

$$\pi_{EMPLOYEE}\,(T1 \bowtie T2)$$

Mixed Fragmentation

- Hybrid (or mixed) fragmentation combines vertical and horizontal fragmentation-first splitting a table by columns, then dividing those fragments by rows (or vice versa). It uses both **SELECT (σ)** and **PROJECT (π)** operations. This is useful when simple horizontal or vertical fragmentation isn't sufficient for data distribution.
- Mixed fragmentation can be done in two different ways:
- The first method is to first create a set or group of horizontal fragments and then create vertical fragments from one or more of the horizontal fragments.
- The second method is to first create a set or group of vertical fragments and then create horizontal fragments from one or more of the vertical fragments.

- The original relation can be obtained by the combination of JOIN and UNION operations which is given as follows:
- $\sigma_P(\pi_{a1,\ a2..,an}(T))$
  $\pi_{a1,a2....,an}\ (\sigma_p(T))$
- **For example**, for our **EMPLOYEE** table, below is the implementation of mixed fragmentation is $\boldsymbol{\pi_{Ename,\ Design}\ (\sigma_{Eno\ <\ 104}(EMPLOYEE))}$
- The result of this fragmentation is:

Ename	Design
A	abc
B	abc
C	abc

- **Advantages:**
- As the data is stored close to the usage site, the efficiency of the database system will increase
- Local query optimization methods are sufficient for some queries as the data is available locally
- In order to maintain the security and privacy of the database system, fragmentation is advantageous
- **Disadvantages:**
- Access speeds may be very high if data from different fragments are needed
- If we are using recursive fragmentation ”repeated fragmentation; A table is fragmented, then each fragment be fragmented again using another fragmentation rule”, then it will be very expensive

Think

- What is fragmentation in DDBMS?

A Backing up data

B Compressing data

C Splitting tables into smaller pieces

D Removing redundancy

- In horizontal fragmentation, how is a table divided?

A By columns

B By rows

C By schemas

D By keys

- What operation is used to reconstruct data from vertical fragments?

A INTERSECT

B UNION

C DIFFERENCE

D JOIN

Data Replication

- Data Replication in DBMS refers to the process of storing copies of the same data at multiple sites or nodes within a distributed database system. The main purpose of replication is to improve data availability, reliability, performance and fault tolerance. In a distributed database environment, data replication ensures that:
- Users can access relevant data locally,
- The system remains operational even if some sites fail and
- The workload is balanced across servers.
- To simplify disaster recovery and backup processes.
- **Note:** It involves copying data from one database (known as the source or publisher) to another (known as the replica, subscriber or target) so that all users can access consistent and up-to-date information without conflicts.

Types of Data Replication

1. Transactional Replication
2. Snapshot Replication
3. Merge Replication
4. Master-Slave Replication
5. Multi-Master Replication
6. Peer-to-Peer Replication

1. Transactional Replication

- Transactional replication involves sending an initial full copy of the database to subscribers, followed by real-time updates as changes occur at the publisher.
- Changes are propagated in the same order as they happen, ensuring transactional consistency.
- It is ideal for server-to-server environments that require high accuracy and reliability.
- **Example:** Banking systems or e-commerce sites that demand consistent and up-to-date transaction data.

2. Snapshot Replication

- Snapshot replication copies and distributes data exactly as it exists at a specific point in time, without tracking changes.
- It sends the entire dataset to subscribers periodically.
- Suitable for databases where data changes infrequently.
- **Drawback:** Can be slower because it replicates the entire dataset each time.

3. Merge Replication

- Merge replication allows both the publisher and subscriber to make changes independently.
- Changes are later merged into a single database.
- It is complex but useful in mobile or client-server environments, where users work offline and synchronize later.
- **Example:** Field data collection systems or distributed applications where clients update data locally.

4. Master-Slave Replication

- In this architecture:
- One database server is designated as the master and
- One or more servers are designated as slaves.
- **Note:** The master handles all write operations and the slaves receive read-only copies from the master.
- This model ensures data consistency and simplifies synchronization.

5. Multi-Master Replication

- All servers act as masters, meaning that updates can occur at any node and changes are replicated across all other nodes.
- Suitable for large distributed systems that require high availability and update flexibility.
- However, it requires conflict detection and resolution mechanisms.

6. Peer-to-Peer Replication

- Every server acts as both a master and a slave, with data replicated in a peer-to-peer manner.
- Commonly used in distributed databases where each node can handle both reads and writes.
- Ensures redundancy and high availability.

Replication Type	Idea	How it Works	Advantages	Disadvantages
Transactional Replication	Replicates **each transaction immediately**	Changes (insert, update, delete) are sent to replicas as soon as they occur	Very **high data consistency**, near real-time updates	More **network traffic** and system overhead
Snapshot Replication	Replicates a **full copy of data at a specific time**	System takes a **snapshot** of the database and sends it to replicas	**Simple implementation**	Data may become **outdated between snapshots**
Merge Replication	Allows **multiple sites to update data independently**	Later the system **merges changes** and resolves conflicts	Good for **offline systems and distributed users**	**Conflict resolution is complex**
Master–Slave Replication	One **master node controls updates**	Master handles all writes, slaves only read replicated data	**Simple and consistent**	If the **master fails**, updates stop
Multi-Master Replication	**Multiple nodes act as masters**	All nodes can read and write data	**High availability and scalability**	**Conflicts may occur** between nodes
Peer-to-Peer Replication	All nodes are **equal peers**	Each node replicates data to others	**Balanced workload and high availability**	**Complex synchronization**

Replication Schemes

1. Full Replication
2. No Replication
3. Partial Replication

1. Full Replication

- In full replication, the entire database is replicated at every site.
- Offers maximum availability and faster local access.
- However, it increases storage requirements and update complexity.
- **Advantages:**

High data availability

Faster query execution (local access)

Improved performance for global queries

- **Disadvantages:**

Difficult concurrency management

Slow update propagation

High storage and network costs

Complex consistency maintenance

No Replication

- In no replication, each data item is stored at only one location in the distributed system.
- **Advantages:**

Reduced concurrency issues

Simplified recovery and management

- **Disadvantages:**

Poor data availability (single point of failure)

Slower query response due to centralized access

Partial Replication

- In partial replication, only selected fragments of the database are replicated based on their importance or access frequency. Balances between storage efficiency and availability.
- **Advantages:**

Optimized storage

Flexibility based on data importance

Better performance for critical data

- **Disadvantages:**

Complex management and configuration

Possible inconsistent data access across sites

Think

- What is replication in distributed databases?

A Dividing tables into fragments

B Linking databases via joins

C Copying data to multiple sites

D Encrypting data

Data Transfer Costs of Distributed Query Processing

- Suppose that the EMPLOYEE and DEPARTMENT relations are distributed at two sites.
- We will assume in this example that neither relation is fragmented.
- The size of the EMPLOYEE relation is 100 * 10,000 = 10 power 6 bytes,
- and the size of the DEPARTMENT relation is 35 * 100 = 3,500 bytes.
- Consider the query Q: For each employee, retrieve the employee name and the name of the department for which the employee works.
- This can be st

Q: $\pi_{Fname,Lname,Dname}(EMPLOYEE \bowtie_{Dno=Dnumber} DEPARTMENT)$

Site 1:

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno

10,000 records
each record is 100 bytes long
Ssn field is 9 bytes long
Dno field is 4 bytes long
Fname field is 15 bytes long
Lname field is 15 bytes long

Site 2:

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date

100 records
each record is 35 bytes long
Dnumber field is 4 bytes long
Mgr_ssn field is 9 bytes long
Dname field is 10 bytes long

Site 1: 10000 row, row size= 100 bytes
Site 2: 100 row, row size=35 bytes
Row in query result= 40 bytes

- The result of this query will include 10,000 records, assuming that every employee is related to a department.
- Suppose that each record in the query result is 40 bytes long.
- The query is submitted at a distinct site 3, which is called the result site because the query result is needed there.
- Neither the EMPLOYEE nor the DEPARTMENT relations reside at site 3.

Solution

- The main objective is to minimize the data transefere.
- There are three simple strategies for executing this distributed query:
- 1. Transfer both the EMPLOYEE and the DEPARTMENT relations to the result site, and perform the join at site 3.
- In this case, a total of 1,000,000 + 3,500 = 1,003,500 bytes must be transferred.

- 2. Transfer the EMPLOYEE relation to site 2 “1000000 bytes”,
- execute the join at site 2,
- and send the result to site 3.
- The size of the query result is 40 * 10,000 = 400,000 bytes,
- so 400,000 + 1,000,000 = 1,400,000 bytes must be transferred.

- 3. Transfer the DEPARTMENT relation to site 1 “3500 bytes”,
- execute the join at site 1, and send the result to site 3.
- In this case, 400,000 + 3,500 = 403,500 bytes must be transferred.
- If minimizing the amount of data transfer is our optimization criterion, we should choose strategy 3.

- Now consider another query Q′: For each department, retrieve the department name and the name of the department manager. This can be stated as follows in the relational algebra:

$$Q': \pi_{Fname,Lname,Dname}(DEPARTMENT \bowtie_{Mgr_ssn=Ssn} EMPLOYEE)$$

- Again, suppose that the query is submitted at site 3.
- The same three strategies for executing query Q apply to Q′, except that the result of Q′ includes only 100 records,
- assuming that each department has a manager:

1. Transfer both the EMPLOYEE and the DEPARTMENT relations to the result

site, and perform the join at site 3. In this case, a total of 1,000,000 + 3,500 =

1,003,500 bytes must be transferred.

2. Transfer the EMPLOYEE relation to site 2, execute the join at site 2, and send the result to site 3. The size of the query result is 40 * 100 = 4,000 bytes, so 4,000 + 1,000,000 = 1,004,000 bytes must be transferred.

3. Transfer the DEPARTMENT relation to site 1, execute the join at site 1, and

send the result to site 3. In this case, 4,000 + 3,500 = 7,500 bytes must be

transferred.

Again, we would choose strategy. The preceding three strategies are the most obvious ones for the case where the result site (site 3) is different from all the sites that contain files involved in the query (sites 1 and 2).

However, suppose that the result site is site 2;

then we have two simple strategies:

1. Transfer the EMPLOYEE relation to site 2, execute the query, and present the

result to the user at site 2. Here, the same number of bytes—1,000,000—must be transferred for both Q and Q′.

2. Transfer the DEPARTMENT relation to site 1, execute the query at site 1, and send the result back to site 2. In this case 400,000 + 3,500 = 403,500 bytes

must be transferred for Q and 4,000 + 3,500 = 7,500 bytes for Q′.

A more complex strategy, which sometimes works better than these simple strategies,

uses an operation called **semijoin**.